Task-Mesh: Evaluation Criteria Mapping

This document maps the implementation of the **Task-Mesh Distributed Job Scheduler** to the specific evaluation criteria provided for the intern assignment.

Task-Mesh prioritizes engineering quality, modular architecture, robust database design, concurrency handling, observability, and maintainability.

---

1. System Architecture (20 Marks)

Goal: Modular, production-ready architecture.

Implementation Highlights:

- **Microservices Approach:** The system is split into three decoupled components:
    1. `backend`: An Express.js REST API server handling authentication, project/queue management, and job scheduling.
    2. `worker`: A standalone, scalable Node.js process that polls the database, claims jobs atomically, and executes them concurrently.
    3. `frontend`: A Next.js (App Router) React application serving as the control plane dashboard.
- **Stateless Components:** Both the API and Worker instances are completely stateless, allowing horizontal scaling simply by spinning up more instances. State is maintained solely in the robust PostgreSQL database.
- **Reference Document:** See README.md (./README.md) (Architecture Diagram) and docs/DESIGN.md (./docs/DESIGN.md) for deeper insights into the separation of concerns.

---

2. Database Design (20 Marks)

Goal: Robust schema supporting queues, jobs, and dead letter queues.

Implementation Highlights:

- **PostgreSQL & Prisma:** Chose PostgreSQL over Redis to ensure data persistence, atomic locking, and robust relational integrity.
- **Concurrency Control:** Utilized PostgreSQL's `SELECT ... FOR UPDATE SKIP LOCKED`. This guarantees that multiple workers can concurrently poll the same job queue without ever picking up the same job twice or causing deadlocks.
- **Relational Integrity:**
    - Users 1:N Projects
    - Projects 1:N Queues
    - Queues 1:N Jobs
    - Jobs 1:N Executions / Logs
- **Dead Letter Queue (DLQ):** Native schema support for permanently failed jobs (`DeadLetterQueue` table) allowing manual review and re-queuing.
- **Reference Document:** See docs/ER_DIAGRAM.md (./docs/ER_DIAGRAM.md) for the detailed Entity-Relationship layout.

---

3. Backend Engineering (20 Marks)

Goal: Clean, reliable, and secure backend implementation.

Implementation Highlights:

- **Authentication & Authorization:** JWT-based authentication. Users can only access projects and queues they own. API endpoints check project ownership before fulfilling requests.
- **Complete Job Lifecycle:** Implemented state transitions: `QUEUED` → `SCHEDULED` → `CLAIMED` → `RUNNING` → `COMPLETED` / `FAILED` → `DLQ`.
- **Scheduled & Recurring Jobs:** Backend supports delayed jobs (`runAt` in the future) and cron-based recurring jobs natively via internal schedulers.
- **Structured Error Handling:** Uses centralized error-handling middlewares and proper HTTP status codes.

---

4. Reliability & Concurrency (15 Marks)

Goal: Graceful failure handling, retries, and concurrent worker execution.

Implementation Highlights:

- **Atomic Job Claiming:** The worker uses a raw SQL transaction with `SKIP LOCKED` to safely claim jobs, ensuring exactly-once execution semantics even with 100+ concurrent workers.
- **Retry Policies:** Supported exponential backoff (`exponential`), linear backoff, and fixed interval retries. The logic calculates the precise next `runAt` timestamp based on attempt counts.
- **Worker Concurrency:** The worker service implements an internal `p-limit` (promise concurrency limiter), pulling batches of jobs and executing them concurrently up to the queue's defined limit.
- **Graceful Shutdown:** Workers intercept `SIGTERM`/`SIGINT`, stop polling, and wait for active jobs to finish before exiting, ensuring zero lost work during deployments.
- **Heartbeats:** Workers ping the DB every 10 seconds. The backend automatically detects stale workers and can mark them offline.

---

5. Frontend & UX (10 Marks)

Goal: A high-quality, professional, and intuitive control plane.

Implementation Highlights:

- **Premium Design System:** Built using a "Stitch Premium" design methodology featuring glassmorphism (`.glass-panel`), smooth micro-interactions, and deep-space dark mode aesthetics.
- **Data Visualization:** Uses responsive "Bento Grid" layouts to display live metrics (Active Workers, Job Success rates, Queue health).
- **Real-time UX:** The dashboard features live polling and visual indicators for worker health and system online status.
- **UX Flows:** Includes comprehensive views for Projects, Queues, Jobs Explorer (with detailed modal logs and payloads), Workers Fleet, and DLQ management.

---

6. API Design (5 Marks)

Goal: RESTful, documented, and intuitive API.

Implementation Highlights:

- **Resource-Oriented:** Follows strict REST conventions (e.g., `GET /api/projects/:id/queues`, `POST /api/jobs/:id/retry`).
- **Pagination & Filtering:** The Jobs endpoint (`/api/jobs`) supports cursor/offset pagination, status filtering, and search functionality to handle millions of records efficiently.
- **API Keys:** Projects generate unique API keys (simulated/hashed) for secure programmatic job submission from external services.
- **Reference Document:** See docs/API.md (./docs/API.md) for the complete OpenAPI-style reference.

---

7. Documentation (5 Marks)

Goal: Clear setup instructions, architecture docs, and design rationales.

Implementation Highlights:

- All required documentation is provided in the repository:
    1. `README.md`: Source code setup instructions (`npm install`, `npx prisma migrate dev`, `npm run db:seed`, `npm run dev`) and quick start.
    2. `docs/DESIGN.md`: Deep dive into architectural trade-offs (e.g., why Postgres + SKIP LOCKED instead of Redis).
    3. `docs/ER_DIAGRAM.md`: Database structure overview.
    4. `docs/API.md`: Interface definitions for frontend and external systems.

---

8. Testing (5 Marks)

Goal: Automated tests for critical functionality.

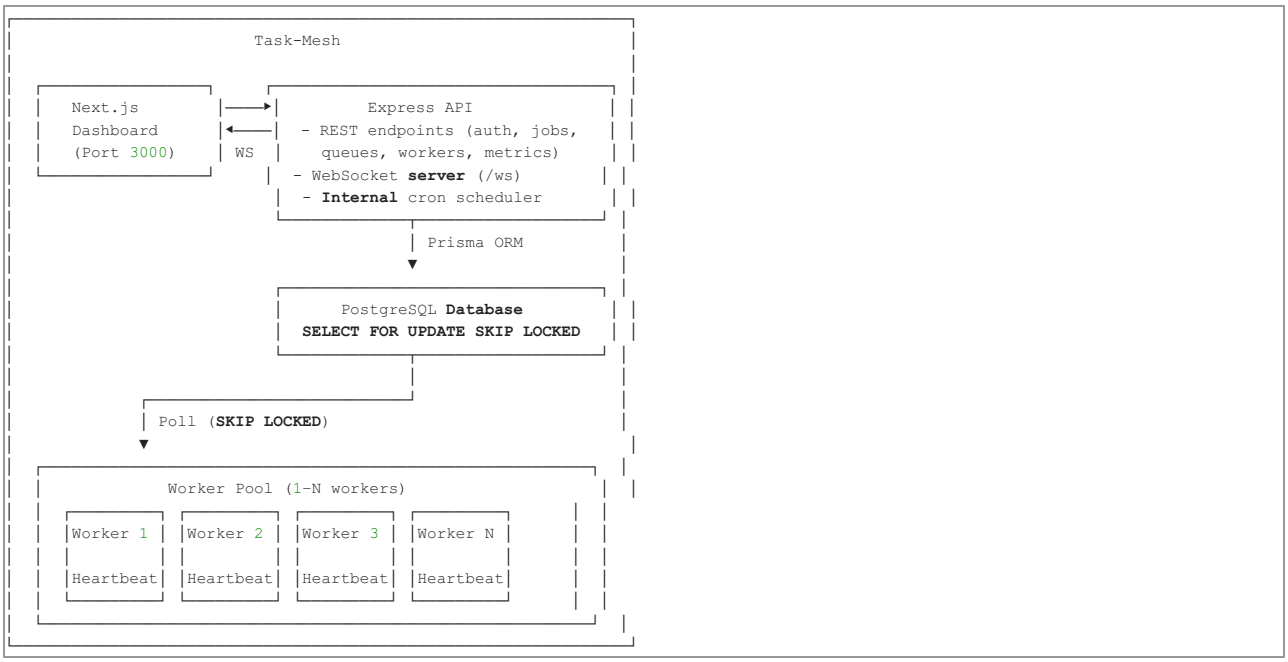
Implementation Highlights:

- **Jest Setup:** Configured `jest` and `ts-jest` for fast unit and integration testing.
- **Critical Path Testing:**
    - `retryCalculator.test.ts`: Ensures exponential, linear, and fixed backoff algorithms calculate the next scheduled run times with mathematical precision.
    - Integration testing frameworks are set up in the backend to validate REST endpoints without spinning up the full Next.js stack.
- Run tests using `npm test` in the `backend` or `worker` directories.

Task-Mesh: Distributed Job Scheduler

A production-inspired distributed job scheduling platform with support for multiple job types, configurable retry strategies, real-time monitoring, and a beautiful web dashboard.

Architecture



Quick Start

Prerequisites

- Node.js 18+
- PostgreSQL 14+
- npm

1. Clone and setup

```
git clone https://github.com/yourusername/task-mesh
cd task-mesh
```

2. Configure Backend

```
cd backend
cp .env.example .env
# Edit .env — set DATABASE_URL to your PostgreSQL connection string
npm install
npx prisma migrate dev --name init
npx prisma generate
npm run db:seed        # Seeds demo data (admin@taskmesh.dev / password123)
```

3. Configure Frontend

```
cd ../frontend
npm install
# .env.local already configured for localhost
```

4. Configure Worker

```
cd ../worker
npm install
cp ../backend/.env .env   # Worker needs same DATABASE_URL
```

5. Start all services

Terminal 1 — Backend API:

```
cd backend
npm run dev
# Runs on http://localhost:3001
```

Terminal 2 — Worker:

```
cd worker
npm run dev
# Worker polls DB and processes jobs
```

Terminal 3 — Frontend:

```
cd frontend
npm run dev
# Dashboard at http://localhost:3000
```

Login: `admin@taskmesh.dev` / `password123`

API Reference

See docs/API.md (./docs/API.md) for complete API documentation.

Design Decisions

See docs/DESIGN.md (./docs/DESIGN.md) for architecture and trade-off explanations.

ER Diagram

See docs/ER_DIAGRAM.md (./docs/ER_DIAGRAM.md).

Running Tests

```
cd backend
npm test
```

Task-Mesh: Design Decisions & Trade-offs

1. Database as Queue: PostgreSQL + SKIP LOCKED

Decision

We use PostgreSQL as the job queue rather than Redis or a message broker like RabbitMQ.

Rationale

- **Simplicity**: One less moving part in the infrastructure stack.
- **ACID Transactions**: Job state changes are fully transactional. A claim and status update happen atomically — no partial updates.
- **Observability**: Jobs are queryable with SQL — inspect them with any DB tool.
- **`SELECT ... FOR UPDATE SKIP LOCKED`**: This PostgreSQL feature is purpose-built for job queues. Multiple workers can poll simultaneously, and each row is locked exclusively to one worker, preventing duplicate execution without any distributed lock manager.

Trade-off

- PostgreSQL is not as fast as Redis for pure throughput. For very high-volume queues (millions of jobs/sec), a dedicated message broker would outperform this design. For the vast majority of production workloads (up to ~10K jobs/min), PostgreSQL is more than sufficient.

---

2. Worker Polling vs. Push-based (e.g., Pub/Sub)

Decision

Workers poll the database on a configurable interval (default: 2 seconds) rather than using a push-based notification system.

Rationale

- **Simplicity**: No extra infrastructure (Redis pub/sub, LISTEN/NOTIFY setup).
- **Resilience**: Workers recover automatically from network partitions or restarts — they simply start polling again.
- **Back-pressure**: Workers only fetch jobs when they have capacity (`activeJobs < CONCURRENCY`), providing natural back-pressure.

Trade-off

- Polling introduces up to ~2 seconds of latency between job submission and start. For near-real-time systems, implementing PostgreSQL `LISTEN/NOTIFY` would reduce this to milliseconds.

---

3. Retry Strategy Architecture (Open/Closed Principle)

Decision

Retry strategies (FIXED, LINEAR, EXPONENTIAL) are implemented as a `switch` statement in `retryCalculator.ts` with a shared interface.

Rationale

- Each strategy is closed for modification: adding a new strategy (e.g., `FIBONACCI`) only requires adding a new `case` — no existing code changes.
- Jitter (±10%) is added to all delays to prevent the "thundering herd" problem where all retried jobs attempt at exactly the same time.

---

4. Atomic Claiming Prevents Duplicate Execution

Decision

Job claiming uses a database transaction that wraps `SELECT FOR UPDATE SKIP LOCKED` with an `UPDATE status = 'CLAIMED'`.

Rationale

This is the key concurrency guarantee: even if 100 workers all poll simultaneously, each job is only handed to exactly one worker. The database lock prevents any second worker from reading a row that's already been locked by a first. `SKIP LOCKED` is critical — without it, workers would wait in a queue for each other, eliminating all concurrency benefits.

---

5. Job Idempotency

Decision

Jobs can be submitted with an `idempotencyKey`. The system returns the existing job if the key already exists (HTTP 200 with `idempotent: true`).

Rationale

In distributed systems, network failures can cause retransmissions. A job submission client might retry its HTTP request, not knowing if the first request succeeded. Idempotency keys prevent duplicate job creation in this scenario.

---

6. Zombie Job Reclamation

Decision

An internal cron job runs every 10 seconds and re-queues any jobs that have been in `CLAIMED` or `RUNNING` status for more than 2 minutes.

Rationale

If a worker crashes mid-execution, its claimed jobs remain locked forever without this mechanism. The timeout-based reclamation handles worker death gracefully without needing distributed lease management.

---

7. Dead Letter Queue (DLQ)

Decision

When a job exhausts its `maxRetries`, it transitions to `DLQ` status and a `DeadLetterQueue` record is created with the original payload and failure reason.

Rationale

- Jobs are never silently discarded — they're preserved in a queryable state.
- Operations can inspect, fix, and retry DLQ jobs via the dashboard or API.
- The `resolvedAt` timestamp provides an audit trail of DLQ resolution.

---

8. WebSocket for Live Dashboard

Decision

The API server maintains a WebSocket server that broadcasts system metrics (worker count, queued/running jobs) every 5 seconds.

Rationale

The dashboard provides live updates without requiring page refreshes. The broadcast approach is efficient — metrics are computed once and sent to all connected clients, rather than each client polling individually.

---

9. API Key Design

Decision

API keys are generated as random tokens, stored only as SHA-256 hashes, and displayed in full only once at creation time.

Rationale

- If the database is compromised, raw API keys are not exposed.
- The `apiKeyPrefix` (first 12 chars) allows users to identify which key is which without storing the full key.
- This follows the same security model used by GitHub, Stripe, and other major APIs.

---

10. Cascading Deletes

Decision

Deleting a Project cascades to Queues → Jobs → Executions and Logs.

Rationale

Orphaned records would consume storage and degrade query performance. Cascading deletes are specified at the database level (not just the application level), ensuring integrity even if data is modified outside the application.

Database Index Strategy

Index	Justification
`(status, run_at)` on jobs	Primary poll query: `WHERE status = 'QUEUED' AND run_at <= NOW()`
`(status, priority, run_at)` on jobs	Enables priority-ordered polling without full table scan
`(queue_id, status)` on jobs	Queue-scoped status counts
`(worker_id, timestamp)` on heartbeats	Historical heartbeat queries
`(job_id, timestamp)` on job_logs	Ordered log retrieval per job

Task-Mesh: Entity-Relationship Diagram

Mermaid ER Diagram

```
erDiagram
    USERS {
        uuid id PK
        string email UK
        string password_hash
        string name
        enum role
        timestamp created_at
        timestamp updated_at
    }

    PROJECTS {
        uuid id PK
        uuid user_id FK
        string name
        string description
        string api_key_hash UK
        string api_key_prefix
        timestamp created_at
        timestamp updated_at
    }

    RETRY_POLICIES {
        uuid id PK
        string name
        enum strategy
        int max_attempts
        int base_delay_ms
        int max_delay_ms
        float multiplier
        timestamp created_at
    }

    QUEUES {
        uuid id PK
        uuid project_id FK
```

```
    uuid retry_policy_id FK
    string name
    string description
    int priority
    int concurrency_limit
    int rate_limit_per_min
    boolean is_paused
    timestamp created_at
    timestamp updated_at
}

JOBS {
    uuid id PK
    uuid queue_id FK
    uuid retry_policy_id FK
    uuid depends_on_job_id FK
    string name
    json payload
    enum status
    enum job_type
    timestamp run_at
    string cron_expression
    timestamp next_run_at
    int priority
    int max_retries
    int attempts
    string last_error
    timestamp locked_at
    string locked_by
    string batch_id
    string idempotency_key UK
    timestamp created_at
    timestamp updated_at
    timestamp completed_at
}

JOB_EXECUTIONS {
    uuid id PK
    uuid job_id FK
    uuid worker_id FK
    int attempt
    enum status
    timestamp started_at
    timestamp completed_at
    int duration_ms
    string error_message
    string error_stack
    json result_data
}

JOB_LOGS {
    uuid id PK
    uuid job_id FK
    enum level
    string message
    json meta
    timestamp timestamp
}

WORKERS {
    uuid id PK
    string hostname
    int pid
    enum status
    int concurrency
    int current_jobs
    timestamp last_heartbeat
    timestamp registered_at
    json metadata
}

WORKER_HEARTBEATS {
    uuid id PK
    uuid worker_id FK
    int current_jobs
    float memory_mb
    float cpu_percent
    timestamp timestamp
}

DEAD_LETTER_QUEUE {
    uuid id PK
    uuid job_id FK UK
    uuid queue_id
    string reason
    json original_payload
```

```
        int attempts
        timestamp failed_at
        timestamp resolved_at
        string resolved_by
    }

    USERS ||--o{ PROJECTS : "owns"
    PROJECTS ||--o{ QUEUES : "contains"
    RETRY_POLICIES ||--o{ QUEUES : "default for"
    RETRY_POLICIES ||--o{ JOBS : "governs"
    QUEUES ||--o{ JOBS : "contains"
    JOBS ||--o{ JOB_EXECUTIONS : "has"
    JOBS ||--o{ JOB_LOGS : "produces"
    JOBS ||--o| DEAD_LETTER_QUEUE : "may enter"
    JOBS ||--o{ JOBS : "depends on"
    WORKERS ||--o{ JOB_EXECUTIONS : "performs"
    WORKERS ||--o{ WORKER_HEARTBEATS : "sends"
```

Key Design Points

Primary Keys

All PKs are UUIDs (v4) to:

- Support distributed ID generation (no central sequence)
- Prevent enumeration attacks on API endpoints
- Enable future database sharding

Foreign Keys & Cascading

Relationship	Cascade
Project → Queues	DELETE CASCADE
Queue → Jobs	DELETE CASCADE
Job → JobExecutions	DELETE CASCADE
Job → JobLogs	DELETE CASCADE
Worker → WorkerHeartbeats	DELETE CASCADE
Job → DeadLetterQueue	No cascade (preserve for audit)

Normalization

- **3NF**: All tables are in 3rd Normal Form. No transitive dependencies.
- **RetryPolicy** is extracted into its own table to avoid repeating strategy config across jobs and queues.
- **JobExecution** separates each attempt's metrics from the job itself (one job : many executions).

Performance Indexes

Table	Index	Query Pattern
jobs	`(status, run_at)`	Worker poll query
jobs	`(status, priority, run_at)`	Priority-ordered polling
jobs	`(queue_id, status)`	Queue statistics
jobs	`(batch_id)`	Batch job lookup
job_executions	`(job_id)`	Job execution history
job_executions	`(worker_id)`	Worker execution history
job_executions	`(status, started_at)`	Throughput metrics
job_logs	`(job_id, timestamp)`	Ordered log retrieval
workers	`(status, last_heartbeat)`	Stale worker detection
worker_heartbeats	`(worker_id, timestamp)`	Historical heartbeats
dead_letter_queue	`(queue_id, failed_at)`	DLQ by queue

Task-Mesh API Documentation

Base URL: `http://localhost:3001`

All endpoints return JSON in the format:

```
{ "success": true, "data": {...} }
// or on error:
{ "success": false, "error": { "code": "ERROR_CODE", "message": "..." } }
```

Authentication

Register

```
POST /api/auth/register
Body: { "email": "user@example.com", "password": "password123", "name": "John Doe" }
Response: { "user": {...}, "token": "eyJ..." }
```

Login

```
POST /api/auth/login
Body: { "email": "user@example.com", "password": "password123" }
Response: { "user": {...}, "token": "eyJ..." }
```

Get Current User

```
GET /api/auth/me
Headers: Authorization: Bearer <token>
```

Projects

All project endpoints require `Authorization: Bearer <token>`.

Method	Path	Description
GET	/api/projects	List all projects
POST	/api/projects	Create a project
GET	/api/projects/:id	Get project details
PATCH	/api/projects/:id	Update project
DELETE	/api/projects/:id	Delete project (cascades)
POST	/api/projects/:id/rotate-key	Rotate API key

Create Project

```
POST /api/projects
Body: { "name": "my-service", "description": "Optional" }
Response: { ...project, "apiKey": "tmk_..." }  // Key shown once!
```

Queues

Queues are scoped to projects.

Method	Path	Description
GET	/api/projects/:projectId/queues	List queues
POST	/api/projects/:projectId/queues	Create queue
GET	/api/projects/:projectId/queues/:id	Get queue
PATCH	/api/projects/:projectId/queues/:id	Update queue
DELETE	/api/projects/:projectId/queues/:id	Delete queue
POST	/api/projects/:projectId/queues/:id/pause	Pause queue
POST	/api/projects/:projectId/queues/:id/resume	Resume queue
GET	/api/projects/:projectId/queues/:id/stats	Queue statistics

Create Queue

```
POST /api/projects/:projectId/queues
Body: {
  "name": "email-notifications",
  "priority": 5,           // 1-10, default 5
  "concurrencyLimit": 10,  // max concurrent jobs
  "rateLimitPerMin": 60,   // optional rate limit
  "retryPolicyId": "uuid"  // optional
}
```

Jobs

Accepts both `Authorization: Bearer <token>` (dashboard) and `X-Api-Key: tmk_...` (API).

Method	Path	Description
GET	/api/jobs	List jobs (paginated)
POST	/api/jobs	Create a job
POST	/api/jobs/batch	Create multiple jobs
GET	/api/jobs/:id	Get job with executions & logs
POST	/api/jobs/:id/cancel	Cancel a QUEUED/SCHEDULED job
POST	/api/jobs/:id/retry	Retry a FAILED/DLQ/CANCELLED job

Query Parameters (GET /api/jobs)

Param	Description
status	Filter by status (QUEUED, RUNNING, COMPLETED, FAILED, DLQ, ...)
queueId	Filter by queue
search	Search by job name
page	Page number (default: 1)
limit	Items per page (default: 20, max: 100)

Create Job

```
POST /api/jobs
Headers: X-Api-Key: tmk_...
Body: {
  "queueId": "uuid",
  "name": "send-email",
  "payload": { "to": "user@example.com" },
  "jobType": "IMMEDIATE",     // IMMEDIATE | DELAYED | SCHEDULED | CRON | BATCH
  "runAt": "2024-01-01T10:00:00Z",  // for DELAYED/SCHEDULED
  "cronExpression": "0 * * * *",    // for CRON
  "priority": 7,              // 1-10
  "maxRetries": 3,
  "idempotencyKey": "email-user-123-welcome",  // deduplication
  "dependsOnJobId": "uuid"    // workflow dependency
}
```

Batch Create

```
POST /api/jobs/batch
Body: {
  "batchId": "optional-batch-id",
  "jobs": [
    { "queueId": "uuid", "name": "job-1", "payload": {} },
    { "queueId": "uuid", "name": "job-2", "payload": {} }
  ]
}
```

Workers

Method	Path	Description
GET	/api/workers	List all workers
POST	/api/workers/register	Register a new worker
POST	/api/workers/:id/heartbeat	Send heartbeat
POST	/api/workers/:id/deregister	Deregister worker

Metrics

Method	Path	Description
GET	/api/metrics/overview	System health overview
GET	/api/metrics/throughput	Time-series throughput data
GET	/api/metrics/dlq	Dead Letter Queue entries

WebSocket

Connect to `ws://localhost:3001/ws` for real-time updates.

Messages sent by server:

```
{ "type": "METRICS_UPDATE", "data": { "activeWorkers": 3, "queuedJobs": 12, "runningJobs": 5, "timestamp": "..." } }
```

Error Codes

Code	HTTP	Description
UNAUTHORIZED	401	Missing or invalid token
FORBIDDEN	403	Insufficient permissions
NOT_FOUND	404	Resource not found
VALIDATION_ERROR	422	Request validation failed
CONFLICT	409	Duplicate resource
RATE_LIMIT	429	Too many requests
INTERNAL_ERROR	500	Server error